

PA1 Report: RTSSP and SHP

Diogo Jesus¹, Nuno Nogueira²

¹ n° 59969.

² n° 60599.

Keywords: RTSSP, SHP, secure streaming, UDP transport security, authenticated encryption, message integrity, replay protection, ECDSA signatures, ECDH key agreement, public key certificates.

1 Introduction

This project focuses on securing a UDP-based real-time media streaming system. In the initial version, the Streaming Server sent MPEG-4 encoded movie frames directly to the Proxy as UDP datagrams, which left the stream exposed to packet sniffing, tampering, and replay attacks.

The solution was developed in two parts. Part 1 introduces RTSSP, a secure streaming layer that protects each UDP datagram while preserving the original streaming behaviour. Part 2 extends this design with SHP, a secure handshake protocol used to authenticate both endpoints, negotiate the cryptographic configuration, and derive fresh keying material before the RTSSP stream begins.

This report describes the implemented packet format, cryptographic configuration, security properties, handshake design, integration between SHP and RTSSP, and the tests used to validate the solution.

2 Part 1: RTSSP Secure Streaming Protocol

2.1 Design and Packet Format

RTSSP protects each UDP datagram by wrapping it in a small protocol header followed by a protected payload. The header has 5 bytes and contains the packet type, protocol version, and payload length:

TYPE (1 B)	VERSION (2 B)	LENGTH (2 B)	payload ...
0x17	0x00 0x03	payload bytes	...

Before protection, the plaintext payload starts with an 8-byte sequence number, followed by the media frame itself. This sequence number is used later by the receiver to detect duplicated packets.

The protected payload depends on the selected cipher suite:

- **AEAD** (GCM, ChaCha20-Poly1305): `encrypt(seqNum[8] | data)`. Authentication is handled internally by the cipher.
- **MAC**: `encrypt(seqNum[8] | data) | MAC(ciphertext)`. The MAC is verified before decryption, following an Encrypt-then-MAC approach.
- **Hash**: `encrypt(seqNum[8] | data | hash(data))`. The hash is carried inside the ciphertext.

With this structure, the Streaming Server sends protected UDP datagrams to the Proxy. The Proxy validates and decrypts them, then forwards the recovered media frames to the local media player.

2.2 Cryptographic Configuration and Implementation

The cryptographic behaviour is defined by a key-value configuration file. The relevant fields are `CONFIDENTIALITY`, which defines the cipher algorithm, `SEED`, which contains the hex-encoded key material, `KEY_SIZE`, which is optional and expressed in bits, `INTEGRITY`, which can be `NULL`, `MAC`, or `H`, and the algorithm-specific fields `MAC` and `H`.

The configured key material is not used directly as an encryption or MAC key. Instead, `CipherSuiteFactory` derives the required keys from `SEED` using HKDF with SHA-256. Domain separation is applied through different HKDF info strings, namely `cipher-key` for encryption and `mac-key` for MAC keys.

This design is also reused in Part 2. In the static RTSSP configuration, the seed comes from the configuration file. In the SHP-based configuration, the ECDH shared secret becomes the seed, allowing both parts to use the same key derivation path.

2.3 Security Properties

RTSSP provides confidentiality by encrypting every media frame before transmission. This prevents an attacker observing the network from directly reading the streamed content.

Integrity and data-origin authentication are provided by AEAD modes or by an explicit MAC. Packets that fail verification are silently dropped and are not forwarded to the media player.

Replay protection is based on the 64-bit sequence number included inside the protected payload. The receiver keeps track of the sequence numbers it has already processed and rejects duplicates. This prevents an attacker from replaying previously captured packets as if they were new.

2.4 Tests and Results

The RTSSP implementation was validated with unit tests covering the configuration parser, cipher suite construction, packet protection, packet recovery, replay rejection, and tampering rejection. The fixed-configuration tests cover 35 configuration files, while the shared-secret tests cover additional configurations where the key material is derived dynamically.

The full streaming pipeline was also executed with `movies/cars.dat`. The Streaming Server completed the transmission successfully and produced the following output:

```
DONE! all frames sent: 30515
Movie duration 128 s
Throughput 238 fps
Throughput 2509 Kbps
```

During this run, the Proxy received, processed, and forwarded the protected stream without reported errors.

3 Part 2: SHP Secure Handshake Protocol

3.1 Design Overview

SHP is a three-message handshake that runs before the RTSSP stream starts. Its purpose is to replace static cryptographic configuration with a dynamic setup phase. During this phase, the Proxy and the Streaming Server authenticate each other, agree on a cipher suite, and derive fresh keying material for the RTSSP session.

The protocol provides mutual authentication using X.509 certificates and ECDSA signatures. It also provides forward secrecy through ephemeral ECDH key pairs generated for each handshake. Freshness is enforced with nonces exchanged by both sides and confirmed through challenge-response values.

3.2 Message Formats and Protocol Flow

SHP exchanges three serialised `ShpMessage` objects:

1. **CLIENT_HELLO**: sent by the Proxy. It contains the requested movie name, the client certificate, an ephemeral ECDH public key, the list of supported cipher suites, a 16-byte nonce, and an ECDSA signature over the message fields.
2. **SERVER_HELLO**: sent by the Streaming Server. It contains the server certificate, an ephemeral ECDH public key, the selected cipher suite, a 16-byte server nonce, the response `clientNonce+1`, and an ECDSA signature over the message fields.
3. **CSSP** (Client Session Setup Payload): sent by the Proxy after deriving the session keys. It contains `encrypt(serverNonce+1|udpPort)`, optionally protected with an integrity proof, using the negotiated session configuration.

After the Hello messages, both sides compute the same ECDH shared secret using their private key and the peer's public key. This shared secret is then passed to `CipherSuiteFactory.fromConfig(suite, sharedSecret)`, producing the `CipherSuite` used for the CSSP message and for the following RTSSP stream.

3.3 Certificates, ECDSA and ECDH

Both parties use a PKCS12 keystore containing their private key and X.509 certificate, and a PKCS12 truststore containing the peer certificate. These stores are generated by `scripts/generate-shp-stores.sh` using `keytool`, `secp256r1` (P-256) keys, and `SHA256withECDSA`, provided by BouncyCastle.

During the handshake, each party signs its Hello message with its private key. The peer verifies the signature using the certificate included in the message, and then checks whether that certificate is trusted by looking it up in the local truststore through `CertificateUtils.isTrusted`.

Key agreement uses ephemeral ECDH through `EcdhKeyAgreement` on `secp256r1`. Each `ShpServer` and `ShpClient` instance creates a fresh `ShpCryptoSpec`, which generates a new ephemeral key pair. The ECDH private key is discarded after the handshake, so compromising a long-term signing key does not expose past session keys.

Cipher suite negotiation follows the server's preference order. The Proxy advertises the cipher suites it supports, and the Streaming Server selects the first compatible suite from its own ordered list. These server-side preference lists are configured per movie, allowing different media resources to use different cryptographic policies.

3.4 Integration with RTSSP

The `CipherSuite` produced by SHP is passed directly to `SecureDatagramSocket` or `SecureMulticastSocket`. From that point on, the RTSSP framing and packet protection are the same as in Part 1.

The main difference is that the session no longer depends on a static `crypto.properties` file. The suite configuration is defined in `cipher-suites.conf`, while the actual key material is derived at runtime from the ECDH shared secret established during SHP.

3.5 Tests and Results

SHP was validated using several test classes in `lib/src/test/java/shp/`. `ShpTest` runs an end-to-end handshake between an in-process client and server, and then uses the resulting `CipherSuite` to perform a full RTSSP send/receive roundtrip. `ShpSuiteNegotiationTest` checks suite selection with different client and server preference lists, including cases

where no common suite exists. `ShpMessagesTest` verifies serialisation roundtrips for all three message types. `ShpCryptoSpecTest` checks ECDSA signing and verification, shared secret agreement, and nonce size.

The streaming phase produced the expected completion output and no errors were reported. Since SHP only runs before the stream starts, its cost affects session setup but not the per-packet RTSSP processing during the movie transmission.

Handshake latency was measured using `ShpHandshakeTimingTest`, with 20 complete handshakes per cipher suite. Representative results are shown in Table 1.

Cipher Suite	Min (ms)	Avg (ms)	Max (ms)
AES/GCM/NoPadding	55	72	121
ChaCha20-Poly1305	55	70	118
AES/CBC/PKCS5Padding + HMAC-SHA256	56	71	120

Table 1: SHP handshake latency over 20 iterations per suite (loopback, secp256r1).

The results are very similar across the three suites. This is expected because the dominant cost of the handshake is the asymmetric cryptography, mainly ECDH and ECDSA, while the symmetric encryption used during SHP only protects a small CSSP payload.

4 Limitations and Assumptions

The trust model follows the assignment assumptions. The endpoints, JVM, operating system, and BouncyCastle provider are considered trusted, and the threat model is limited to attacks against the communication channel.

The implementation does not support session resumption, certificate revocation, or multiple simultaneous SHP clients. The truststore-based model is sufficient for this project because both parties are pre-configured with each other’s certificate, but it would need to be extended for a larger deployment.

5 Conclusion

This project implements two complementary security layers for a UDP-based real-time media streaming system. Part 1 introduces RTSSP, which protects each streamed frame with confidentiality, integrity, data-origin authentication, and replay protection. Part 2 extends this with SHP, a lightweight handshake protocol that adds mutual authentication, ephemeral key agreement, and dynamic cipher suite negotiation before the stream begins.

The implementation was validated with unit tests covering the cryptographic components, packet protection logic, SHP message handling, cipher suite negotiation, and end-to-end integration. Manual streaming executions also confirmed that the negotiated security configuration can be used to deliver the protected movie stream successfully.